

Resumo da tela do Seu Valdir

Guia para entender o HTML, o CSS, o JavaScript e justificar as decisões

1. Ideia principal

A página funciona como um pequeno aplicativo de celular para calcular uma corrida. O fluxo é simples: informar os preços, iniciar o trajeto, acrescentar cobranças e mostrar o total.

Necessidade da persona	Decisão tomada
Celular distante dos olhos	Letras grandes, botões altos e poucos elementos por linha.
Luz do sol na lateral	Contraste forte e alternância entre modo diurno e noturno.
Uso no suporte do painel	Interface estreita, em coluna e adaptada ao formato de celular.
Mostrar o preço ao passageiro	Valor final destacado com tamanho muito maior que os demais textos.

2. HTML: a estrutura

O HTML define o conteúdo e a ordem da tela. Ele foi dividido em três seções: trajeto, cobranças adicionais e resumo final.

- **Trajeto:** origem, destino, distância, tempo e os campos de preço.
- **Adicionais:** botões para pedágio e espera.
- **Resumo:** valor final e botão para começar outra corrida.
- Os **id** identificam os elementos para que o JavaScript possa encontrá-los.
- **aria-label**, **aria-live** e **aria-pressed** melhoram o uso com leitores de tela.

Por que o campo usa texto?

Os preços usam **type="text"** com **inputmode="decimal"**. Assim, o celular mostra um teclado adequado para números e o campo aceita a vírgula usada em valores brasileiros.

3. CSS: aparência e adaptação

O CSS transforma a estrutura em uma interface legível e confortável para toque. As cores são guardadas em variáveis, o que facilita trocar o tema inteiro.

- Largura máxima de **430 px**: simula um celular no computador e ocupa toda a largura no telefone.
- Fonte base de **18 px**: melhora a leitura com o aparelho distante.
- Botões com pelo menos **60 px**: oferecem uma área de toque maior.
- Elementos em uma coluna: evita controles apertados e facilita acompanhar o fluxo.
- Amarelo: remete ao táxi e destaca as ações e o total.
- Contraste e bordas fortes: ajudam na leitura em ambientes iluminados.
- **prefers-reduced-motion**: remove transições quando o usuário prefere menos movimento.

Mapa simulado

O mapa é apenas uma representação visual feita com CSS. Quando o JavaScript adiciona a classe **em-viagem**, a posição do táxi muda do ponto A para o ponto B.

Modo noturno

A classe **noturno** troca os valores das variáveis de cor. Isso evita repetir todas as regras e mantém o desenho da tela igual nos dois temas.

4. JavaScript: estado da aplicação

Três variáveis guardam o que está acontecendo:

```
let total = 0;
let corridaIniciada = false;
let modoNoturno = false;
```

- **total**: guarda o preço acumulado.
- **corridaIniciada**: impede cobranças antes do início.
- **modoNoturno**: registra qual tema está ativo.

document.querySelector

Procura cada elemento pelo id. Depois disso, o código consegue mudar textos, adicionar classes, bloquear campos e ler os valores digitados.

5. Funções e decisões

mostrarTotal()

Formata o total como moeda brasileira. Por isso o resultado aparece como **R\$ 25,50**, em vez de um número cru como 25.5.

iniciarCorrida()

- Lê os valores por quilômetro e por minuto.
- Troca a vírgula por ponto, porque o JavaScript interpreta decimais com ponto.
- Converte os textos em números e exige valores maiores que zero.
- Calcula: **valor do km x 8 + valor do minuto x 20**.
- Bloqueia os campos depois do início para evitar alterações acidentais.
- Atualiza o status, a animação, as mensagens e o valor final.

adicionarPedagio() e adicionarEspera()

Cada função verifica se a corrida começou. Se começou, soma a cobrança e atualiza a tela. Caso contrário, orienta o usuário a iniciar o trajeto primeiro.

alternarTema()

Adiciona ou remove a classe **noturno**, troca o texto do botão e atualiza **aria-pressed** para comunicar o estado do controle.

comecarDeNovo()

Volta o total a zero, libera e limpa os campos, devolve o mapa ao início e muda o status para PARADO. Isso permite atender uma nova corrida sem recarregar a página.

Por que a função fica sem parênteses?

Em `addEventListener("click", iniciarCorrida)`, o nome fica sem parênteses porque a função deve ser entregue ao navegador para execução futura. Com `iniciarCorrida()`, ela seria executada imediatamente durante o carregamento.

6. Fala pronta para apresentação

Eu construí a tela pensando no Seu Valdir, que usa o celular longe dos olhos e sofre com a luz do sol. Por isso usei letras grandes, botões altos, contraste forte e modo noturno. O HTML organiza as informações, o CSS adapta a tela ao formato de celular e o JavaScript calcula a corrida e responde aos cliques. Os campos aceitam vírgula porque esse é o formato usado no Brasil. Depois que a corrida começa, eles são bloqueados para evitar alteração acidental. O valor final fica grande para ser mostrado ao passageiro.

7. Limitações que você pode reconhecer

- O trajeto usa valores fixos de **8 km** e **20 minutos**; não existe integração com GPS.
- A tela não consegue confirmar se o carro está realmente parado.
- A animação representa o deslocamento, mas não calcula uma rota real.
- Se forem digitadas letras, o valor pode virar **NaN** e a validação atual não exibe uma mensagem adequada.

Resumo em uma frase

Cada decisão tenta equilibrar a persona, a facilidade de uso no celular e a restrição de usar somente HTML, CSS e JavaScript básico.